

Decision Records

Understanding Why Those Decisions Were Made!

Presented by Sarah “sadukie” Dutkiewicz

Presented at:



2026

Common QA Frustrations



The Wasted Investigation



The Break in Automation



The Blame Game



The Compliance Crisis

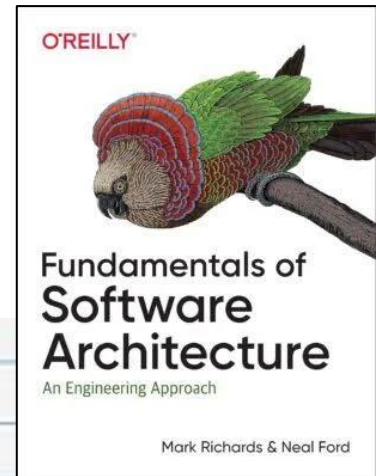


What are decision records?



A written record
capturing the
intricacies of a decision

An Example of a Decision Record



ADR Format from *Fundamentals of Software Architecture*

TITLE

Identifier + short description

STATUS

Draft, Proposed, Accepted, Superseded, Obsoleted

CONTEXT

What is causing us to make this decision?

DECISION

What is the decision and its justification

CONSEQUENCES

What is the impact? What are the **trade-offs**?

COMPLIANCE

How do we ensure people comply with this decision?

NOTES

Additional supporting information

These headings are a **general recommendation.**

Evolve your template for **your organization's needs.**

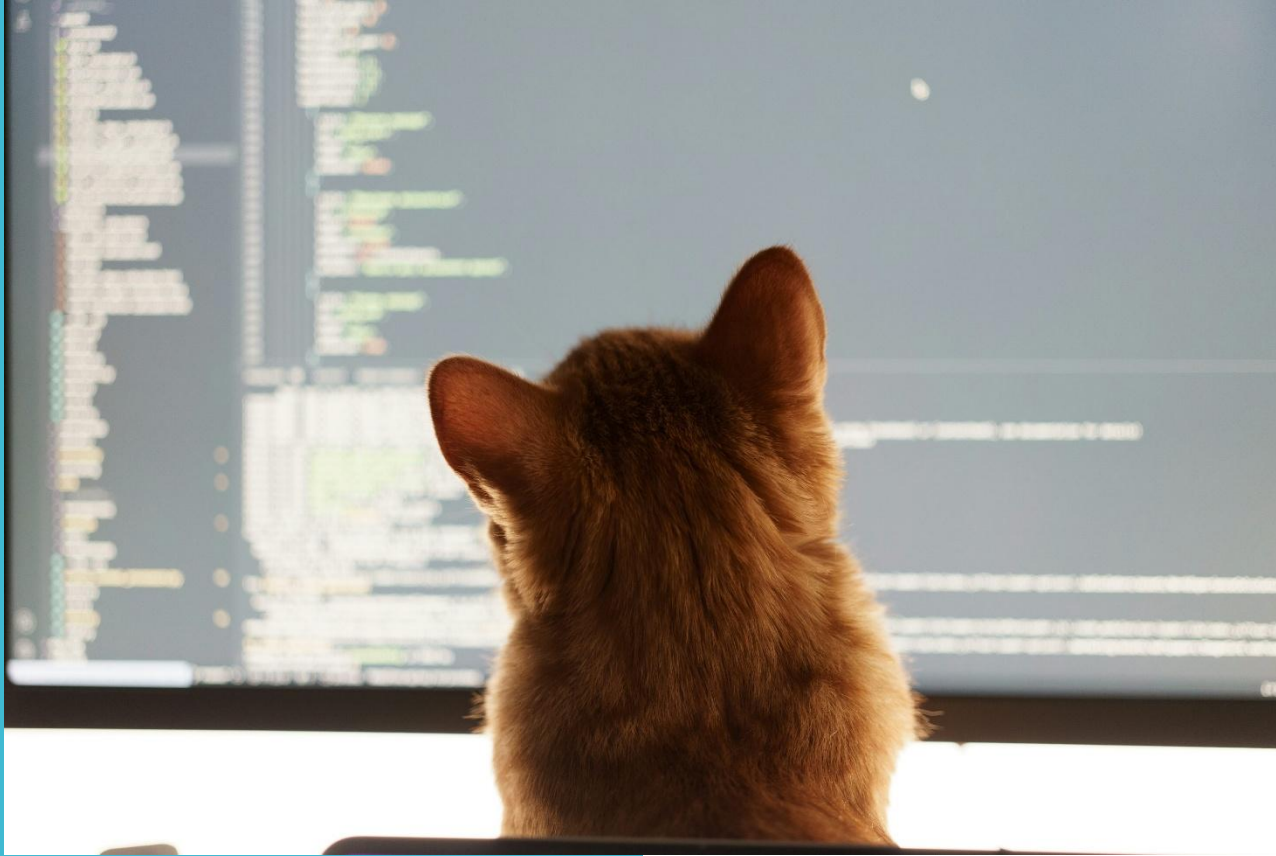
- Sadukie

Contents of a Decision Record – Michael T. Nygard's Template

- **Title**
 - Identifier and short description – typically short noun phrases
- **Status**
 - Draft, Proposed, Accepted, Superseded, Obsoleted
- **Context**
 - What are the forces at play? Why do we even need to make this decision?
- **Decision**
 - What was decided? This is commonly written with “We will...” statements.
- **Consequences**
 - What are the trade-offs? Capture positive, negative, and neutral consequences.
- **Additional Notes**
 - From compliance guidance to additional supporting information



Who are decision records for?



Anyone who is
onboarding
into a
codebase



Stakeholders



Decision Makers





Future You!



Which decisions should be captured?



All of Them?!

But that feels like it's too much!



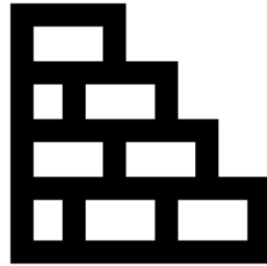


It Depends!

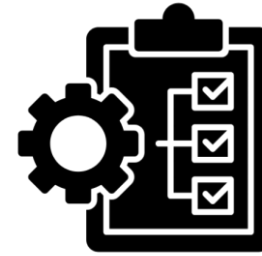


Architectural significance of a decision

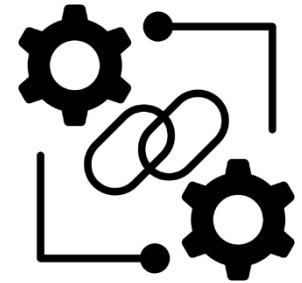
Factors of Architectural Significance



Structure



Architectural
Characteristics



Dependencies



Interfaces



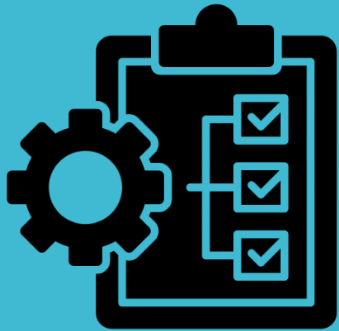
Construction

Architectural Significance – Structure



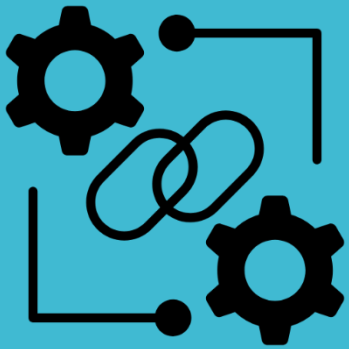
- Impact patterns or styles of architecture
- Examples:
 - Modular Monolith, Clean Architecture, Vertical Slices?
 - Modules in a modular monolith
 - Bounded contexts in microservices

Architectural Significance – Architectural Characteristics



- Impacts any architectural characteristic / “-ility”
- Examples:
 - Security
 - Performance
 - Maintainability
 - Scalability
 - Auditability
 - Usability

Architectural Significance - Dependencies



- Impacts coupling between components
- Examples:
 - Direct Database Access vs ORM
 - Shared Libraries
 - API – Async vs Sync

Architectural Significance – Interfaces



- Impacts how services and components are accessed
- Examples:
 - REST vs GraphQL
 - Protocols – gRPC vs HTTP
 - Event Streaming

Architectural Significance - Construction



- Anything important to the construction of the solution
- Examples:
 - Platforms
 - Frameworks
 - Tools
 - Processes



When should decisions be captured?

When you have the context



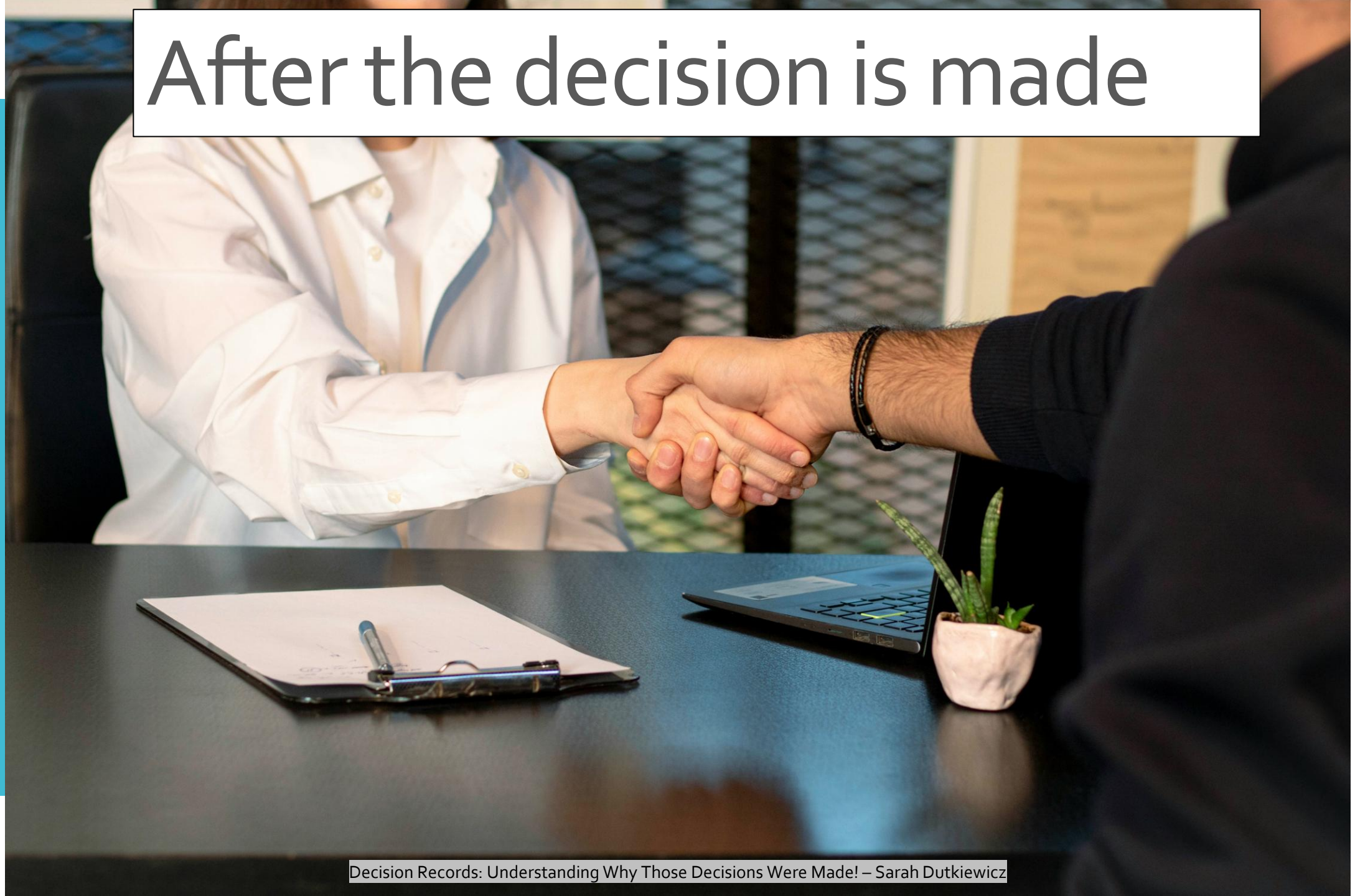
Before the decision is finalized



As the decision is made



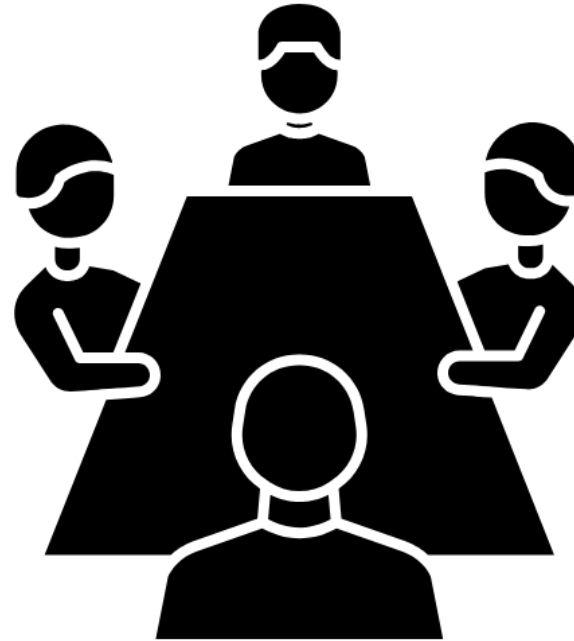
After the decision is made



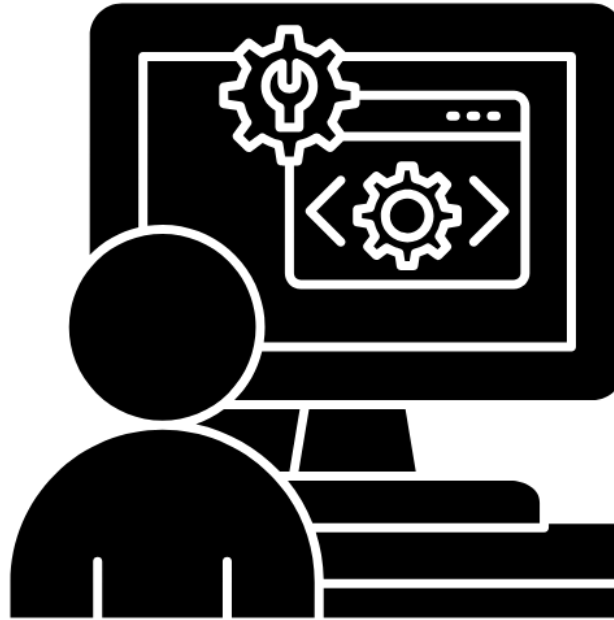


Who should capture these decisions?

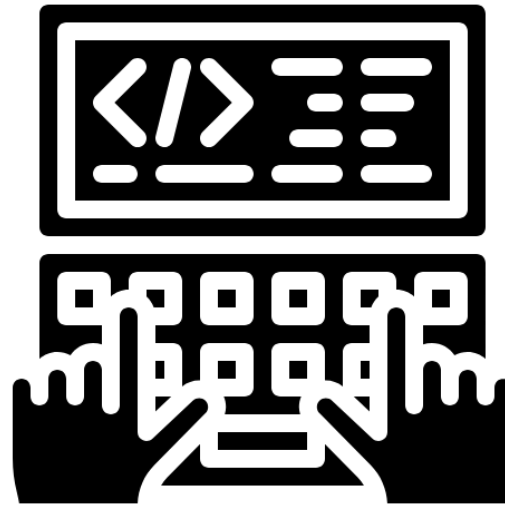
Architecture Review Board



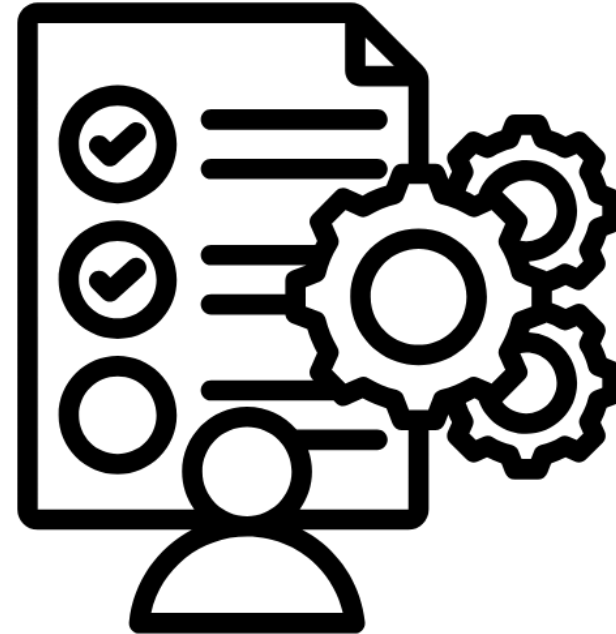
Solution Architects & Other Software Architects



Developers



Project Manager



Someone
who is
good at
creating
documentation



Someone
who is
good with
AI!





Where should decisions be captured?

Where to Store Decision Records

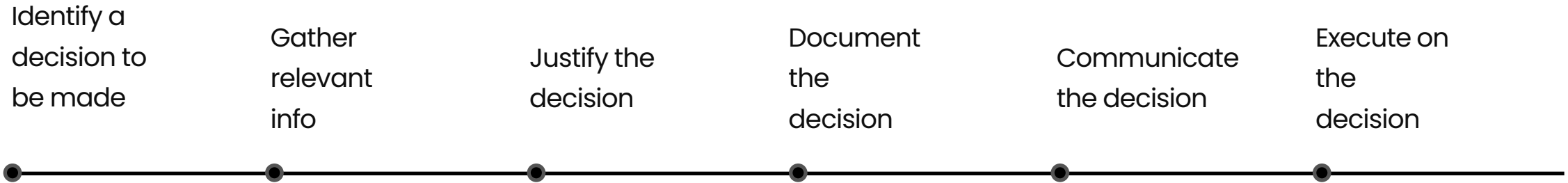
- In a document store
- In a formal document control system
- In a folder on an intranet
- In a wiki
- **In a place where all who are impacted can review the decision records**



How are decisions captured?

Decision Records

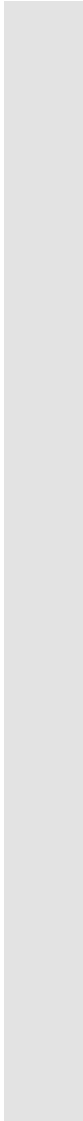
Decision Process



Can be more formalized with Architecture Decision Committees, commonly seen in enterprise architecture organizations



Examples of Decisions to Capture



Example: Security with MFA Options

ADR 003 - Security Decision: Multi-Factor Authentication (MFA)

Date - 2025-11-13

Status

Active

Context

We are establishing a strategy for multi-factor authentication (MFA) to enhance the security of our application and protect against unauthorized access.

Decision

We will implement Multi-Factor Authentication (MFA) requiring at least two factors for all user accounts.

Options Considered

- **Option 1: 2FA via Email & SMS:** Utilizing email and SMS messages as the primary second factor for authentication.
- **Option 2: 2FA via Email & SMS with Time-Based One-Time Passwords (TOTP):** Employing TOTP generated via authenticator apps.
- **Option 3: Full MFA with Biometrics, Authenticator Apps, and Passkeys:** Implementing a comprehensive MFA strategy utilizing biometrics, authenticator apps, and passkeys.

Rationale

- **Security Risks of Email & SMS:** Email and SMS-based 2FA are vulnerable to phishing attacks, SIM swapping, and other interception methods.
- **Stronger Authentication:** We prioritize a more secure authentication approach by leveraging factors resistant to interception, such as biometrics, authenticator apps (TOTP), and passkeys.
- **Authenticator Apps & Passkeys:** These offer a significantly higher level of security compared to SMS and email.

Consequences

- **Positive:** Significantly reduced risk of unauthorized access, enhanced security posture, and improved compliance.
- **Negative:** Increased complexity for users, potential need for user education and support.

Open Issues

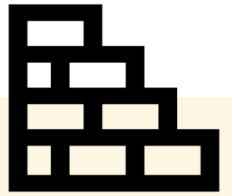
- **User Education:** Developing and providing clear instructions and support for users transitioning to MFA.
- **Passkey Adoption:** Strategizing the rollout and adoption of passkeys as a more secure and user-friendly alternative to traditional authentication methods.



Architectural Characteristics



Example: Modular Monolith vs Clean Architecture



Structure

ADR 001 - Clean Architecture vs. Modular Monolith

Status: Accepted

Context

We were exploring architectural approaches for a new project, specifically considering Clean Architecture versus a Modular Monolith. Early in the process, we lacked a robust understanding of the underlying business domain. Our primary goal was to establish a clear and organized structure for the application.

Decision

We chose to implement Clean Architecture.

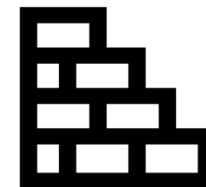
Consequences

This decision resulted in a loosely coupled, testable application with well-defined boundaries, enabling independent development and easier maintenance over the long term. It also prioritized alignment with evolving business requirements due to its separation of concerns.

Additional Notes

This initial decision was driven by a desire for future flexibility and maintainability, acknowledging the potential risks of a monolithic structure given our limited initial domain knowledge.

Example: .NET Version



Structure

ADR 001 - .NET Version Selection

- **Date:** 2025-07-03
- **Status:** Active
- **Priority:** High

Context

We are facing a decision regarding the .NET version to use for our upcoming project. The release of .NET 10 is imminent, alongside continued support for .NET 8 and .NET 9. A key consideration is the long-term support implications and the risk of rework.

Problem Statement

We need to determine the most appropriate .NET version to adopt, balancing the benefits of newer versions with the stability requirements of our project, particularly regarding support lifecycle and potential upgrade paths to .NET 10.

Decision

We will initially adopt .NET 9 and plan to upgrade to .NET 10 as soon as it's mature and ready for production use.

Options Considered

- **Option 1: .NET 8:** This offers continued support until November 10, 2026. However, it's nearing end-of-life, and starting with a nearly-obsolete version is undesirable.
- **Option 2: .NET 9:** Offers a reasonable support window (May 12, 2026) and a solid foundation for our application.
- **Option 3: .NET 10 (Initial):** While .NET 10 will likely launch in November 2025 with 3 years support, the prematurity of the release and lack of thorough evaluation necessitates a cautious approach.

Rationale

- **Minimize Rework:** Starting with .NET 9 avoids committing to a version that will quickly become obsolete.
- **Long-Term Stability:** .NET 9 provides a sufficient support window for our project's foreseeable needs.
- **Flexibility:** Allows us to readily upgrade to .NET 10 when it's stabilized and proven viable, aligning with Microsoft's support patterns.
- **Steve's Insight:** Steve's decision to upgrade to .NET 10 as soon as possible is a key driver.

Consequences

Positive

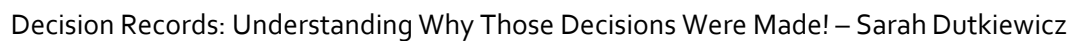
- Reduced risk of needing to rework the application due to .NET version obsolescence.
- Greater flexibility to adopt .NET 10 when it's mature.

Negative

- Potential for some functionality to be dependent on .NET 9 specific features. (Requires thorough assessment of dependencies).
- Potential for rework if there are breaking changes in .NET 10.

Domain Events in C#

Demo of using AI to generate the record



Using AI to Generate Decision Records

- Be explicit
 - Specify the headings in a particular order.
 - Specify the format – so something specific like **Markdown code fence** or **rich-text format**.
 - **As you go through this process, build it in instructions or a skill file!**
- Walk it through your decision
 - Use a model that has **summarize** capabilities for better success.
 - Talk through your decision and let it organize into the desired format.
- Review the generated document
 - Make sure that it caught what you said.
 - Make sure there aren't hallucinations in the generated document.

ADR 001 - Domain Event Implementation

Status - Draft

Context

We're investigating the best approach for generating and handling domain events in our C# project. Several options were considered, ranging from full-fledged event bus frameworks to simpler, more lightweight solutions. The primary goal is to decouple our domain logic from presentation layers and enable asynchronous communication between components. We need a solution that is manageable, well-suited for a project of this scale, and doesn't introduce unnecessary licensing costs.

Decision

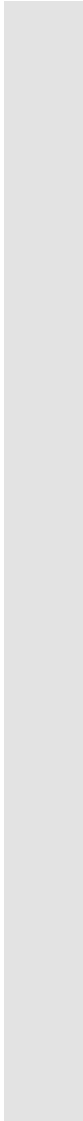
We have decided to implement domain events using **FastEndpoints**.

Consequences

- **Reduced Complexity:** FastEndpoints simplifies the event generation and handling process compared to heavier frameworks like NServiceBus or MediatR.
- **Cost-Effective:** FastEndpoints is open-source and free to use, eliminating licensing concerns that would have been associated with NServiceBus and MediatR.
- **Integration:** FastEndpoints is already being used within this project for managing API endpoints, creating a cohesive development experience. This minimizes the learning curve and integration effort.
- **Suitable Scale:** FastEndpoints is designed for smaller projects like this, offering a sensible balance between features and complexity.
- **Performance:** FastEndpoints' design focuses on performance and efficiency.

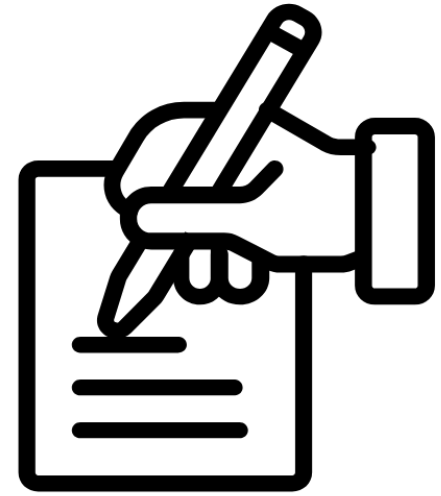
Additional Notes

- We evaluated NServiceBus and MediatR, which provide robust eventing capabilities but carry licensing costs that would likely be prohibitive for this project.
- The decision was made with consideration for the long-term maintainability and operational costs of the eventing infrastructure. A heavier framework would require more overhead for configuration, monitoring, and potential upgrades.
- We will explore FastEndpoints' features (e.g., automatic serializing/deserializing of event data) to optimize our implementation.



In Summary...

Document the
decision...



Because
someone will
ask “why?!”





And that someone
may be YOU!



Additional Resources

Additional Resources

- [adr-tools command line tool](#)
- [*Fundamentals of Software Architecture*](#)
- [Cognitect – Documenting Architecture Decisions \(Nygard\)](#)
- [Joel Parker Henderson's GitHub repo on ADRs](#)
- [Architecture decision record - Microsoft Azure Well-Architected Framework](#)
- [Architecture Decision Records - AWS Prescriptive Guidance](#)

Document
Your Decisions
TODAY!



Sarah “sadukie” Dutkiewicz

 <https://linkedin.com/in/sadukie>